

Nombre:

Tercer Parcial

Paradigmas de la Programación

12/06/2026

ej.1	ej. 2	ej. 3	ej. 4	ej. 5

Programación lógica

Dada la siguiente base de conocimiento:

None

```
trabaja_en(juan, ventas).
trabaja_en(elena, ventas).
trabaja_en(serena, marketing).
trabaja_en(esteban, ventas).
supervisa(serena, juan).
supervisa(serena, elena).
supervisa(esteban, juan).

mismo_area(X, Y) :- trabaja_en(X, A), trabaja_en(Y, A).
jefe(X, Y) :- supervisa(X, Y), mismo_area(X, Y).
colega(X, Y) :- mismo_area(X,Y), not(jefe(X, Y)).
```

Le preguntamos al intérprete de Prolog si `colega(elena, juan)` es True. Para cada uno de los siguientes subobjetivos, indique si el algoritmo de backtracking necesita que sea True para demostrar `colega(elena, juan)`.

subobjetivo	
trabaja_en(elena,ventas).	True False No se comprueba
mismo_area(elena,juan).	True False No se comprueba
mismo_area(juan,elena).	True False No se comprueba
supervisa(elena,juan).	True False No se comprueba

Nombre:

Frameworks

En el siguiente código en Spark, un framework de computación distribuida:

None

```
import org.apache.spark.sql.SparkSession

val spark = SparkSession.builder()
  .appName("ContadorPalabras")
  .master("local[*]")
  .getOrCreate()

val sc = spark.sparkContext

val lineas = sc.textFile("corpus.txt")

val conteos = lineas
  .flatMap(linea => linea.split(" "))
  .map(palabra => (palabra, 1))
  .reduceByKey(_ + _)

conteos.saveAsTextFile("salida/")

spark.stop()
```

- [V / F] La función `linea => linea.split(" ")` pasada a `flatMap` es un hotspot: es un lugar donde el desarrollador introduce lógica específica del problema.
- [V / F] La inversión de control de Spark consiste en que es el framework el que decide en qué nodo y en qué momento ejecutar cada transformación, sin intervención del programador.
- [V / F] El mecanismo de distribución de datos entre nodos, scheduling de tareas y tolerancia a fallos es un hot spot que implementa el desarrollador.
- [V / F] Las funciones pasadas a `flatMap`, `map` y `reduceByKey` son los hotspots del framework: son los puntos de extensión donde el desarrollador conecta su lógica al pipeline distribuido de Spark.

Nombre:

Seguridad en lenguajes de programación

En el siguiente código:

Python

```
def transferir(cuenta_origen, cuenta_destino, monto):
    cuenta_origen["saldo"] -= monto
    cuenta_destino["saldo"] += monto
    return True

def procesar_transferencias(transferencias):
    exitosas = 0
    for t in transferencias:
        resultado = transferir(t["origen"], t["destino"],
t["monto"])
        if resultado:
            exitosas += 1
    print(f"Transferencias exitosas: {exitosas}")
```

Encontramos los siguientes comportamientos inesperados

- Si `t["monto"]` es mayor que el saldo de `t["origen"]`, `transferir`, se descuenta el monto igual y el saldo queda negativo sin ninguna advertencia.
- Si `t["monto"]` es negativo, `transferir` se aumenta el saldo de la cuenta origen y disminuye el de la destino
- `transferir` siempre devuelve `True`, por lo que el conteo de `exitosas` no refleja si la transferencia fue válida o no.
- Si `transferir` lanza una excepción en segunda primera línea, el saldo de `cuenta_destino` queda modificado.

Conscientes de estos comportamientos, esta otra versión del código:

Python

```
def transferir(cuenta_origen, cuenta_destino, monto):
    if monto <= 0:
        raise ValueError(f"El monto debe ser positivo, recibido: {monto}")
    if cuenta_origen["saldo"] < monto:
        raise ValueError(f"Saldo insuficiente: {cuenta_origen['saldo']} < {monto}")
    cuenta_origen["saldo"] -= monto
    cuenta_destino["saldo"] += monto
    return True
```

Nombre:

- [V / F] Implementa programación ofensiva.
- [V / F] Implementa defensa por contrato: valida las precondiciones antes de modificar el estado.
- [V / F] Implementa defensa mediante guardas condicionales.
- [V / F] Implementa defensa mediante manejo de excepciones.
- [V / F] Implementa defensa implementando tipos específicos.

Scripting

En el siguiente código:

Shell

```
1. #!/bin/bash
2.
3. DIRECTORIO=$1
4. EXTENSION=$2
5. DESTINO=$3
6.
7. if [ ! -d "$DIRECTORIO" ]; then
8.     echo "Error: $DIRECTORIO no existe" >&2
9.     exit 1
10. fi
11.
12. archivos=$(find "$DIRECTORIO" -name ".*$EXTENSION" -type f)
13. cantidad=$(echo "$archivos" | wc -l)
14.
15. echo "Encontrados $cantidad archivos .$EXTENSION"
16.
17. mkdir -p "$DESTINO"
18.
19. for archivo in $archivos; do
20.     nombre=$(basename "$archivo")
21.     cp "$archivo" "$DESTINO/$nombre"
22.     echo "Copiado: $nombre"
23. done
24.
25. echo "Listo. $cantidad archivos copiados a $DESTINO"
```

Nombre:

Para cada una de las siguientes propiedades, consigne el número de una línea del programa en la que se encuentre la propiedad, o ninguna si la propiedad no se encuentra en este fragmento de código:

Propiedad	Línea
Seguridad de tipos: los tipos de datos de las variables son suficientemente específicos como para evitar comportamientos inesperados.	
Las variables no requieren asignación explícita de tipo.	
Buena comunicación con el sistema operativo: invoca directamente una herramienta del sistema desde el script.	
Uso de expresiones regulares sofisticadas (cuantificadores, abstracciones sobre conjuntos de caracteres, agrupaciones, negaciones, etc.).	
Reglas de alcance explícitas.	
Construcciones lingüísticas de alto nivel propias del dominio de aplicación.	

Concurrencia

En una primera iteración con un asistente de programación con IA, generamos este fragmento de código:

```
None
var contador = 0

val t = new Thread(() => {
    Thread.sleep(100)
    contador += 1
})
t.start()

val f1 = Future { Thread.sleep(300); "resultado 1" }
val f2 = Future { Thread.sleep(200); "resultado 2" }
val combinado = Future.sequence(List(f1, f2))
t.join()
val resultados = Await.result(combinado, 5.seconds)
println(s"Contador: $contador, Resultados: $resultados")
```

En esta primera versión del código se da una condición de carrera. Solicitamos al asistente explícitamente que evite condiciones de carrera, y el código resultante es el siguiente:

Nombre:

None

```
val contador = new AtomicInteger(0)

val t = new Thread(() => {
    Thread.sleep(100)
    contador.incrementAndGet()
})
t.start()

val f1 = Future { Thread.sleep(300); "resultado 1" }
val f2 = Future { Thread.sleep(200); "resultado 2" }
val combinado = Future.sequence(List(f1, f2))
t.join()
val resultados = Await.result(combinado, 5.seconds)
println(s"Contador: ${contador.get()}, Resultados: $resultados")
```

Asumiendo que esta segunda versión del código efectivamente NO tiene condiciones de carrera (efectivamente no las tiene, pueden asumirlo con tranquilidad):

- [V / F] La sincronización con `join` garantiza que no hay condiciones de carrera.
- [V / F] `incrementAndGet` tiene garantía de atomicidad.
- [V / F] El estado mutable compartido son los Futuros.
- [V / F] El estado mutable compartido es la variable a la que se le puede hacer asignación destructiva.